

Raw-Python Agent vs. LangChain Agent

Architecture Comparison

Both agents follow the same loop: **User** → **Model** → **Tool Call** → **Tool Execution** → **Observation** → **Model** → **Final Answer**. LangChain doesn't introduce a new architecture — it automates the orchestration code Day 1 wrote by hand.

Aspect	Day 1 — Raw Python	Day 2 — LangChain (create_agent)
Model client	Manual API client, raw requests	ChatGroq wrapper with standardized .invoke()
Tools	Plain functions in a dict	@tool-decorated functions; docstring = model-facing description
Reasoning loop	Manually written act/observe/repeat	Handled internally by create_agent
Memory	Manual list, resent each turn	RunnableWithMessageHistory failed (incompatible with unified messages state) — fell back to a manual session dict, same as Day 1
Error handling	try/except around each tool call	Not automatic — still had to catch errors inside each tool, same as Day 1
Structured output	Manual JSON parsing/validation	response_format=<PydanticModel> handles it automatically
Visibility	Fully explicit, one file	Shorter code, but internals hidden inside the framework

Bottom line: LangChain cuts code volume, not conceptual complexity — memory and error handling turned out to be the same responsibility in both, just relocated (and in two cases, silently broken) inside framework internals.

Annotated Reasoning Trace

Query: *"What is the weather in Lahore and Islamabad, and which city is warmer?"*

[ACT] get_weather({'city': 'Lahore'})

[OBSERVE] {"temperature_c": 38, "condition": "Sunny"}

[ACT] get_weather({'city': 'Islamabad'})

[OBSERVE] {"temperature_c": 31, "condition": "Partly cloudy"}

[FINAL] Lahore is warmer — 38°C vs 31°C, a 7°C difference.

- **Reason** — inferred, not printed: the model splits the compound question into two cities before acting. `create_agent` exposes no chain-of-thought text.
- **Act** — two separate, correctly-scoped tool calls instead of one malformed call.
- **Observe** — both results are collected before the model answers.
- **Final** — the 7°C comparison is computed from real tool output, not guessed.

Same act/observe/repeat structure as Day 1's raw ReAct loop, but this trace had to be manually pulled from `.stream(stream_mode="updates")` events, since `create_agent` has no `verbose=True` flag.